

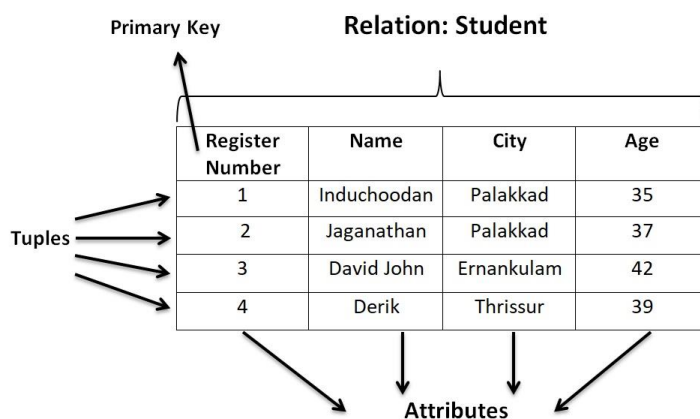
UNIT 1 :

Relational Databases:- Introduction - Purpose of Database System – Database System Applications - View of data: Data Abstraction, Instances and Schemas, Data Models – Database Architecture - Database Users and Administrators: Database Users and Interfaces, DBA – Introduction to the Relational Model: Structure of Relational Database, database Schema, Keys, Relational Query language – The Relational Algebra: Fundamental Operations, Formal definition of the relational algebra, additional relational algebra operations – The Entity Relationship model: Entity Set, Relationship Set, Attributes – Constraints: Mapping cardinalities, Key Constraints, Participation Constraints - E-R Diagrams: Basic structure, Complex attributes, Roles, Non binary relationship sets, Weak Entity Set, Relational Database Design using ER- to Relational Mapping – Extended ER Features: Specialization, Generalization, Attribute inheritance, Constraints on generalization, Aggregation.

Relational Databases: - Introduction

A relational database is a collection of information that organizes data in predefined relationships where data is stored in one or more tables (or "relations") of columns and rows, making it easy to see and understand how different data structures relate to each other. Relationships are a logical connection between different tables, established on the basis of interaction among these tables. Relational databases are useful for any information need where data points both relate to each other and also must be managed in a consistent, secure, rules-based way. This is what makes relational databases most popular for businesses and enterprises.

Table Student



Relation: The table is called as relation here **student** is the name of the relation.

Attributes: The columns in the table represent the attributes. Attributes are the properties that define an entity. Example: **Register Number, Name, City, Age.**

Tuples: Each row in a relation is known as tuple. The above relation contains 4 tuples.

Degree: The number of attributes in the relation is known as the degree of the relation. The student relation defined above has degree 4.

Cardinality: The number of tuples in a relation is known as cardinality. The student relation defined above has cardinality 4.

Primary Keys: Each table has a primary key, a unique identifier for each record.

Foreign Keys: These are used to link tables by referencing the primary key of another table.

The Role of Schemas in Relational Databases

A schema in a relational database can be thought of as a blueprint for how the database is constructed. It outlines the tables, fields, relationships, and constraints within the database. Schemas help ensure data integrity and provide a clear structure to the database.

- **Data Integrity:** Ensures that data entered into the database is accurate and reliable.
- **Relationships:** Defines how tables are connected to one another.
- **Normalization:** Refers to organizing data to reduce redundancy and improve data integrity.

In a schema, we can define various constraints such as unique, not null, and default values for the data fields. These constraints help maintain consistent data and facilitate better queries.

Key Features of Relational Databases

Relational databases come with several advantages that make them widely used for various applications:

- **Flexibility:** Easily adapt to changes like adding new fields or tables.
- **Scalability:** Handle growing data without significant performance loss.
- **ACID Properties:** Ensures transactions are processed reliably. This stands for Atomicity, Consistency, Isolation, and Durability.

Advantages and Disadvantages of Relational Databases

Like any technology, relational databases have their advantages and disadvantages.

Advantages:

- Structured data management helps in data integrity and consistency.
- Robust querying capabilities through SQL.
- Support for complex transactions through ACID properties.

Disadvantages:

- May require more storage due to data redundancy.
- Can become inefficient when handling unstructured data like multimedia files.
- The complexity of managing relationships can increase as the database grows.

Purpose of Database System:

- **Data Storage and Management:** It provides a systematic way to store, retrieve, and manage large amounts of data efficiently.
- **Data Integrity and Security:** Ensures data accuracy and consistency through constraints, and protects data from unauthorized access via security measures.
- **Data Retrieval and Querying:** Allows users to easily query and retrieve data using languages like SQL, enabling complex data operations.
- **Data Relationships:** Manages relationships between different data entities, which helps in organizing data in a relational manner.
- **Concurrent Access:** Supports multiple users to access and manipulate data simultaneously without conflicts.
- **Backup and Recovery:** Provides mechanisms for backing up data and restoring it in case of failures, ensuring data durability.
- **Scalability:** Can grow with increasing data volume and user load, accommodating future needs.
- **Data Analysis:** Facilitates data analysis and reporting, allowing organizations to derive insights from their data.

Overall, a database system is fundamental for effective data management in various applications, from small businesses to large enterprises.

Database System Applications

Relational database systems are widely used across various applications due to their ability to efficiently manage and manipulate structured data. Here are some key applications:

- **Business Applications**

Customer Relationship Management (CRM): Stores customer data, track interactions, and manage sales processes.

Enterprise Resource Planning (ERP): Integrates core business processes, including finance, HR, and supply chain management.

- **E-commerce**

Product Catalog Management: Manages product listings, pricing, and inventory.

Order Processing: Handles customer orders, payment processing, and order history.

- **Financial Systems**

Banking Applications: Manages accounts, transactions, and customer data securely.

Investment Management: Tracks portfolios, trades, and market data.

- **Healthcare**

Electronic Health Records (EHR): Stores patient information, treatment history, and appointment scheduling.

Pharmacy Management: Manages prescriptions, inventory, and patient medication records.

- **Telecommunications**

Billing Systems: Manages customer billing, usage data, and account management.

Network Management: Stores data about network performance and customer usage patterns.

- **Education**

Student Information Systems: Manages student records, grades, and course schedules.

Learning Management Systems (LMS): Stores course materials, student progress, and assessments.

- **Social Media**

User Profiles: Stores user information, posts, and interactions.

Content Management: Manages multimedia content and user-generated data.

- **Government**

Public Records Management: Stores data related to citizens, taxes, and legal documents.

Census Data: Manages demographic information for policy-making and planning.

- **Data Warehousing**

Business Intelligence: Aggregates data from various sources for analysis and reporting.

ETL Processes: Extracts, transforms, and loads data for reporting and analysis.

- **Web Applications**

Content Management Systems (CMS): Manages website content, user data, and interactions.

API Services: Provides structured data access for various applications.

The concepts of data abstraction, instances, and schemas

Data Abstraction

Data abstraction refers to the process of simplifying complex data structures to hide the implementation details while exposing only the necessary features. There are three levels of data abstraction:

- **Physical Level:** This level describes how data is physically stored in the database. It involves the data structures, file formats, and storage mechanisms used.
- **Logical Level:** This level defines what data is stored in the database and the relationships among those data. It focuses on the structure of the data, such as tables, fields, and constraints, without concern for how they are implemented physically.
- **View Level:** This is the highest level of abstraction, which provides a way to present data to users without exposing the underlying complexities. Views can be tailored to meet the needs of different users, allowing for different representations of the same data.

Instances

An instance of a database refers to the actual data stored at a particular moment in time. Each time data is added, modified, or deleted, the instance changes. It represents the state of the database at any specific point, which can vary over time as transactions are executed.

Schemas

A schema is the blueprint of the database that defines its structure. It includes definitions of tables, fields, data types, relationships, and constraints. Schemas can be thought of as the conceptual design of the database, which remains stable, while instances change frequently. There are typically two types of schemas:

- **Database Schema:** It outlines the structure of the entire database, including all tables, fields, and relationships.
- **External Schema (or Views):** These represent how individual users or applications interact with the data, tailored to their specific needs and perspectives.

Data Models in Database Management System

Data models in database management systems (DBMS) define how data is structured, stored, and manipulated. They serve as blueprints for how information is organized and accessed.

Here are some key types of data models:

Hierarchical Model

- **Structure:** Data is organized in a tree-like structure with parent-child relationships.
- **Usage:** Useful for applications with a clear hierarchical relationship, such as organizational structures.

Network Model

- **Structure:** Similar to the hierarchical model but allows more complex relationships with multiple parent nodes.
- **Usage:** Suitable for applications needing many-to-many relationships.

Relational Model

- **Structure:** Data is organized in tables (relations) consisting of rows and columns.
- **Usage:** Most widely used model; supports SQL for querying data.
- **Key Feature:** Data integrity and normalization principles.

Object-Oriented Model

- **Structure:** Data is represented as objects, similar to object-oriented programming.
- **Usage:** Useful for applications requiring complex data types and relationships.

Entity-Relationship Model (ER Model)

- **Structure:** Graphical representation of entities and their relationships.
- **Usage:** Often used in the design phase to outline database schema.

Document Model

- **Structure:** Data is stored in documents, typically JSON or XML format.
- **Usage:** Common in NoSQL databases; flexible schema design.

Key-Value Model

- **Structure:** Data is stored as key-value pairs.
- **Usage:** Efficient for simple lookups and fast data retrieval, often used in caching.

Column-Family Model

- **Structure:** Data is stored in columns rather than rows, allowing for efficient querying of large datasets.
- **Usage:** Common in distributed databases like Apache Cassandra.

Graph Model

- **Structure:** Data is represented as nodes (entities) and edges (relationships).
- **Usage:** Ideal for applications with complex relationships, such as social networks.

Database Architecture

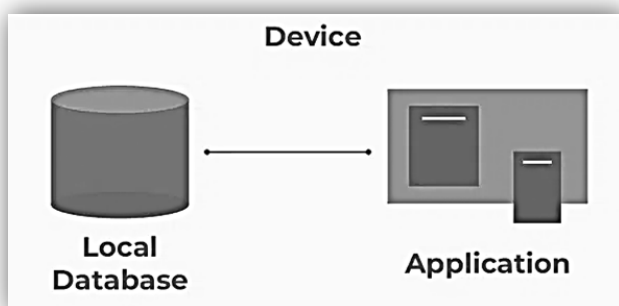
There are several types of DBMS Architecture that we use according to the usage requirements. Types of DBMS Architecture are discussed here.

- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

In 1-Tier Architecture the database is directly available to the user, the user can directly sit on the DBMS and use it that is, the client, server, and Database are all present on the same machine. This setup is simple and is often used in personal or standalone applications where the user interacts directly with the database.

For Example: A Microsoft Excel spread sheet is a great example of one-tier architecture. Everything—the user interface, application logic and data is handled on a single system. The user directly interacts with the application, performs operations like calculations or data entry and stores data locally on the same machine. This architecture is simple and works well for personal, standalone applications where no external server or network connection is needed.



Advantages of 1-Tier Architecture

- **Simple Architecture:** 1-Tier Architecture is the simplest architecture to set up, as only a single machine is required to maintain it.
- **Cost-Effective:** No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.
- **Easy to Implement:** 1-Tier Architecture can be easily deployed, and hence it is mostly used in small projects.

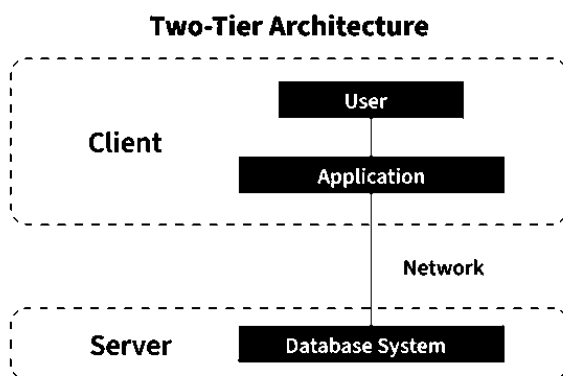
2-Tier Architecture

The 2-tier architecture is similar to a basic client-server model. The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities. On the client side, the user interfaces and application programs are run. The application on the client side establishes a connection with the server side to communicate with the DBMS.

For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.

- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.
- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information, and transaction logs.

The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.



Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.
- **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

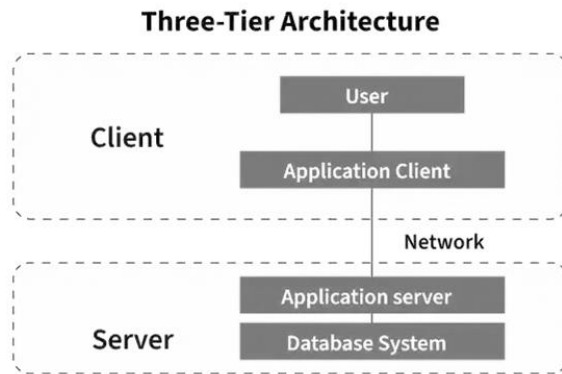
3-Tier Architecture

In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place. This intermediate layer acts as a medium for the

exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.

For Example: E-commerce Store

- **User:** You visit an online store, search for a product and add it to your cart.
- **Processing:** The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database:** The product details, your cart and order history are stored in the database for future reference.



Advantages of 3-Tier Architecture

- **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

Disadvantages of 3-Tier Architecture

- **More Complex:** 3-Tier Architecture is more complex in comparison to 2-Tier Architecture. Communication Points are also doubled in 3-Tier Architecture.
- **Difficult to Interact:** It becomes difficult for this sort of interaction to take place due to the presence of middle layers.

Database Users and Administrators

Database Users

Types of Users:

End Users: Individuals who interact with the database through applications or interfaces.

Power Users: Users with advanced knowledge who can execute complex queries and reports.

Permissions: Users are granted specific permissions to access and manipulate data based on their roles.

Database Interfaces

- **Graphical User Interfaces (GUIs):** Tools like SQL Server Management Studio (SSMS) or MySQL Workbench that provide a visual way to interact with databases.
- **Command-Line Interfaces (CLIs):** Tools like MySQL or PostgreSQL command-line clients that allow users to write SQL commands directly.
- **APIs:** Application Programming Interfaces that allow external applications to interact with the database programmatically.

Database Administrators (DBAs)

Roles and Responsibilities:

- **Installation and Configuration:** Setting up database management systems (DBMS).
- **Performance Monitoring:** Ensuring databases run efficiently and optimizing queries.
- **Backup and Recovery:** Implementing strategies to protect data and restore it in case of failure.
- **Security Management:** Managing user access and ensuring data integrity.
- **Troubleshooting:** Diagnosing and resolving database issues.

Introduction to the Relational Model

The Relational Model represents data and their relationships through a collection of tables. Each table also known as a relation consists of rows and columns. Every column has a unique name and corresponds to a specific attribute, while each row contains a set of related data values representing a real-world entity or relationship. This model is part of the record-based models which structure data in fixed-format records each belonging to a particular type with a defined set of attributes.

Characteristics of the Relational Model

- **Data Representation:** Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
- **Atomic Values:** Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
- **Unique Keys:** Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
- **Attribute Domain:** Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.
- **Tuples as Rows:** Rows in a table, called tuples, represent individual records or instances of real-world entities or relationships.
- **Relation Schema:** A table's structure is defined by its schema, which specifies the table name, attributes, and their domains.
- **Data Independence:** The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
- **Integrity Constraints:** The model enforces rules like:
 - **Domain constraints:** Attribute values must match the specified domain.

- **Entity integrity:** No primary key can have NULL values.
- **Referential integrity:** Foreign keys must match primary keys in the referenced table or be NULL.
- **Relational Operations:** Supports operations like selection, projection, join, union, and intersection, enabling powerful data retrieval manipulation.
- **Data Consistency:** Ensures data consistency through constraints, reducing redundancy and anomalies.
- **Set-Based Representation:** Tables in the relational model are treated as sets, and operations follow mathematical set theory principles

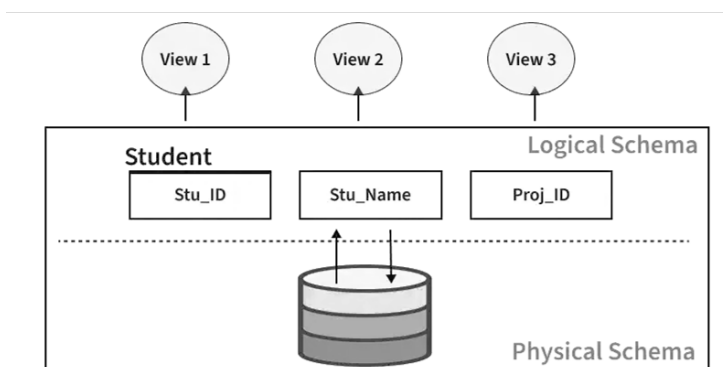
Database Schema

A database schema is the design or structure of a database that defines how data is organized and how different data elements relate to each other. It acts as a blueprint, outlining tables, fields, relationships, and rules that govern the data.

Key points about a database schema:

- It defines how data is logically organized, including tables, fields, and relationships.
- It outlines the relationships between entities, such as primary and foreign keys.
- It helps resolve issues with unstructured data by organizing it in a clear, structured way.
- Database schemas guide how data is accessed, modified, and maintained.

The schema provides the framework that makes it easier to understand, manage, and use data in a database. It's created by database designers to ensure the data is consistent and efficiently organized.



Types of Database Schemas

Physical Database Schema

- A physical schema defines how data is stored in the storage system, including the arrangement of files, indices and other storage structures. It specifies the actual code and syntax needed to create the database structure. Essentially, it determines where and how the data is stored in the physical storage medium.
- The database administrator decides the storage locations and organization of data within the storage blocks. This schema represents the lowest level of abstraction

Logical Database Schema

- A logical database schema defines the logical structure of the data, including tables, views, relationships, and integrity constraints. It describes how data is organized in tables and how the attributes of these tables are connected. The logical schema ensures that the data is stored in an organized manner, while maintaining data integrity.
- Using Entity-Relationship (ER) modeling, the logical schema outlines the relationships between different data components. It also defines integrity constraints to ensure the quality of data during insertion and updates.
- This schema represents a higher level of abstraction compared to the physical schema, focusing on logical constraints and how the data is structured, without dealing with the physical storage details.

Keys in database

Primary Key

Definition: A unique identifier for each record in a table.

Characteristics:

- Must contain unique values.
- Cannot contain NULL values.
- A table can have only one primary key.

Foreign Key

Definition: A field (or collection of fields) in one table that uniquely identifies a row of another table.

Characteristics:

- Establishes a relationship between two tables.
- Can contain NULL values, allowing for optional relationships.

Unique Key

Definition: A constraint that ensures all values in a column are unique.

Characteristics:

- Similar to a primary key but can allow NULL values.
- A table can have multiple unique keys.

Composite Key

Definition: A primary key that consists of two or more columns to uniquely identify a row.

Characteristics:

- Useful for creating a unique identifier when a single column is not sufficient.

Candidate Key

Definition: A column, or set of columns, that can qualify as a potential primary key.

Characteristics:

- Must contain unique values.
- The primary key is chosen from the candidate keys.

Surrogate Key

Definition: An artificial key created for the purpose of uniquely identifying a record, often a sequential number.

Characteristics:

- Not derived from application data.
- Helps maintain the uniqueness of records without relying on natural data.

Natural Key

Definition: A key that is formed from attributes that already exist in the real world.

Characteristics:

- Derived from the data itself, like a Social Security Number.

Relational Query language

Relational Query Language (RQL) is a language designed for querying and manipulating data in relational databases. The most commonly used RQL is SQL (Structured Query Language). Relational Query Language is used by the user to communicate with the database user requests for the information from the database. Relational algebra breaks the user requests and instructs the DBMS to execute the requests. It is the language by which the user communicates with the database. They are generally on a higher level than any other programming language. These relational query languages can be Procedural and Non-Procedural.

Types of Relational Query Language

- Procedural Query Language
- Non-Procedural Language

Procedural Query Language

In Procedural Language, the user instructs the system to perform a series of operations on the database to produce the desired results. Users tell what data to be retrieved from the database and how to retrieve it. Procedural Query Language performs a set of queries instructing the DBMS to perform various transactions in sequence to meet user requests.

Relational Algebra is a Procedural Query Language

Relational Algebra could be defined as the set of operations on relations.

There are a few operators that are used in relational algebra :

- **Select (σ):** Returns rows of the input relation that satisfy the provided predicate.
Notation: $\sigma_{\text{condition}}(\text{Relation})$
Example: $\sigma_{\text{age} > 30}(\text{Employees})$ selects all employees older than 30.
- **Projection (π):** Show the list of those attribute which we desire to appear and rest other attributes are eliminated from the table. It separates the table vertically.
Notation: $\pi_{\text{column1, column2}}(\text{Relation})$
Example: $\pi_{\text{name, age}}(\text{Employees})$ selects the names and ages of all employees.
- **Set Difference (-):** It returns the difference between two relations . If we have two relations R and S then R-S will return all the tuples (row) which are in relation R but not in Relation S.

Notation: $\text{Relation1} - \text{Relation2}$

Example: Employees – Managers finds employees who are not managers.

- **Cartesian Product (X):** Combines every tuple (row) of one table with every tuple (row) in other table ,also referred as cross Product .

Notation: $\text{Relation1} \times \text{Relation2}$

Example: Employees $\times$ Departments pairs each employee with each department.

- **Union (U):** Outputs the union of tuples from both the relations. Duplicate tuples are eliminated automatically. It is a binary operator means it require two operands.

Notation: $\text{Relation1} \cup \text{Relation2}$

Example: Employees $\cup$ Managers combines all employees and managers.

- **Intersection ($\cap$):** Finds tuples that are common to both relations.

Notation: $\text{Relation1} \cap \text{Relation2}$

Example: Employees $\cap$ Managers finds employees who are also managers.

- **Join operation** in relational algebra is used to combine tuples from two or more relations based on a related attribute. Here are the primary types of join operations:

Natural Join ($\bowtie$)

Definition: Combines tuples from two relations where the values of common attributes are equal. It automatically removes duplicate columns.

Notation: $\text{Relation1} \bowtie \text{Relation2}$

Example:

Suppose we have:

Employees: (EmployeeID, Name, DepartmentID)

Departments: (DepartmentID, DepartmentName)

The natural join Employees $\bowtie$ Departments results in:

(EmployeeID, Name, DepartmentID, DepartmentName)

Theta Join (θ)

Definition: Combines tuples from two relations based on a specified condition (θ). The condition can involve any comparison operator (e.g., =, <, >).

Notation: $\text{Relation1} \bowtie_{\text{condition}} \text{Relation2}$

Example:

Using the same relations as above, a theta join like Employees $\bowtie_{\text{Employees.DepartmentID=Departments.DepartmentID}}$ Departments will combine the two relations where DepartmentID matches.

Equi-Join

Definition: A specific case of theta joins where the condition is based on equality between attributes.

Notation: Similar to theta join, but specifically focuses on equality.

Example:

Employees $\bowtie_{\text{Employees.DepartmentID=Departments.DepartmentID}}$ Departments is an equi-join.

Outer Join

Outer joins extend the concept of joins by including tuples that do not have matching values in the other relation. There are three types:

Left Outer Join ($\bowtie$):

Definition: Includes all tuples from the left relation and the matched tuples from the right relation. Non-matching tuples from the right will have nulls.

Notation: Relation1 $\bowtie$ Relation2

Example: Employees $\bowtie$ Departments includes all employees, even those without departments.

Right Outer Join ($\bowtie$):

Definition: Includes all tuples from the right relation and the matched tuples from the left relation. Non-matching tuples from the left will have nulls.

Notation: Relation1 $\bowtie$ Relation2

Example: Departments $\bowtie$ Employees includes all departments, even those without employees.

Full Outer Join ($\bowtie$):

Definition: Includes all tuples from both relations, with nulls in place where there are no matches.

Notation: Relation1 $\bowtie$ Relation2

Example: Employees $\bowtie$ Departments includes all employees and all departments, regardless of matches.

Non-Procedural Language

In Non Procedural Language user outlines the desired information without giving a specific procedure or without telling the steps by step process for attaining the information. It only gives a single Query on one or more tables to get .The user tells what is to be retrieved from the database but does not tell how to accomplish it.

For Example: get the name and the contact number of the student with a Particular ID will have a single query on STUDENT table.

Relational Calculus is a Non Procedural Language.

Relational Calculus exists in two forms:

- **Tuple Relational Calculus (TRC):** Tuple Relational Calculus is a non-procedural query language, It is used for selecting the tuples that satisfy the given condition or predicate. The result of the relation can have one or more tuples (row).
- **Domain Relational Calculus (DRC):** Domain Relational Calculus is a Non Procedural Query Language, the records are filtered based on the domains, DRC uses the list of attributes to be selected from relational based on the condition.

Procedural Language	Non-Procedural Language
It is command-driven language.	It is a function-driven language

It works through the state of machine.	It works through the mathematical functions.
Its semantics are quite tough.	Its semantics are very simple.
It returns only restricted data types and allowed values.	It can return any datatype or value
Overall efficiency is very high.	Overall efficiency is low as compared to Procedural Language.
Size of the program written in Procedural language is large.	Size of the Non-Procedural language programs are small.
It is not suitable for time critical applications.	It is suitable for time critical applications.
Iterative loops and Recursive calls both are used in the Procedural languages.	Recursive calls are used in Non-Procedural languages.

Entity-Relationship (ER) model

The Entity-Relationship (ER) model is a crucial conceptual framework for designing databases in a Relational Database Management System (RDBMS). Here's how the ER model applies to RDBMS:

Key Components of the ER Model in RDBMS

1. Entities and Entity Sets:

- **Entities:** Real-world objects or concepts. Each entity represents a unique instance.
- **Entity Sets:** A collection of similar entities, represented as tables in an RDBMS.
- **Example:** An "Employee" table where each row represents an individual employee.

2. Attributes:

- Characteristics that describe entities.
- **Types of Attributes:**
 - **Simple:** Atomic values (e.g., FirstName, LastName).
 - **Composite:** Composed of multiple attributes (e.g., FullName).
 - **Derived:** Values calculated from other attributes (e.g., Age from BirthDate).

3. Relationships and Relationship Sets:

- **Relationships:** Associations between entities.
- **Relationship Sets:** A collection of similar relationships, typically implemented as foreign keys in RDBMS.
- **Example:** A "Works_For" relationship can be represented using a foreign key in the "Employee" table that references the "Department" table.

Example

Consider a simple ER model with two entities: "Employee" and "Department".

- **Employee Table:**
 - Columns: EmployeeID (PK), FirstName, LastName, DepartmentID (FK)
- **Department Table:**
 - Columns: DepartmentID (PK), DepartmentName
- **Relationship:** An employee belongs to one department, while a department can have many employees (1:N relationship).

Constraints

a. Mapping Cardinalities

- **Definition:** Specifies the number of instances of one entity that can or must be associated with instances of another entity.
- **Types:**
 - **One-to-One (1:1):** Each instance of entity A is related to one instance of entity B.
 - **One-to-Many (1:N):** Each instance of entity A can be related to multiple instances of entity B.
 - **Many-to-One (N:1):** Multiple instances of entity A can relate to a single instance of entity B.
 - **Many-to-Many (M:N):** Instances of entity A can relate to multiple instances of entity B and vice versa.

b. Key Constraints

- **Definition:** Specifies attributes that can uniquely identify an entity within an entity set.
- **Example:** In an "Employee" entity set, the "EmployeeID" could serve as a primary key.

c. Participation Constraints

- **Definition:** Specifies whether all or only some entity instances participate in a relationship.
- **Types:**
 - **Total Participation:** Every instance of the entity set must participate in the relationship (indicated by a double line).
 - **Partial Participation:** Some instances of the entity set may not participate in the relationship.

E-R Diagrams: Basic structure, Complex attributes, Roles, Non binary relationship sets, Weak Entity Set

Entity-Relationship (E-R) diagrams are visual representations of the entities, attributes, and relationships in a database. They help in designing databases by providing a clear and structured view of the system. Here's an overview of key components, including basic structures, complex attributes, roles, non-binary relationships, and weak entity sets.

1. Basic Structure of E-R Diagrams

- **Entities:** Represented by rectangles, entities are objects or concepts (e.g., Employee, Department).
- **Attributes:** Represented by ovals connected to their respective entities. They describe properties of entities (e.g., EmployeeID, EmployeeName).
- **Relationships:** Represented by diamonds, relationships connect entities (e.g., Works_In connecting Employee and Department).

2. Complex Attributes

- **Definition:** Attributes that can be divided into smaller sub-parts, providing more detailed information.
- **Example:** An address can be a complex attribute consisting of Street, City, State, and ZipCode.

- **Representation:** Complex attributes are often shown as ovals connected to their respective components.

3. Roles

- **Definition:** Used to clarify the participation of entities in a relationship, especially when the same entity set participates in multiple roles.
- **Example:** In a Manages relationship, an employee can act as both a manager and a subordinate.
- **Representation:** Roles can be labeled next to the relationship line to indicate the specific role each entity plays.

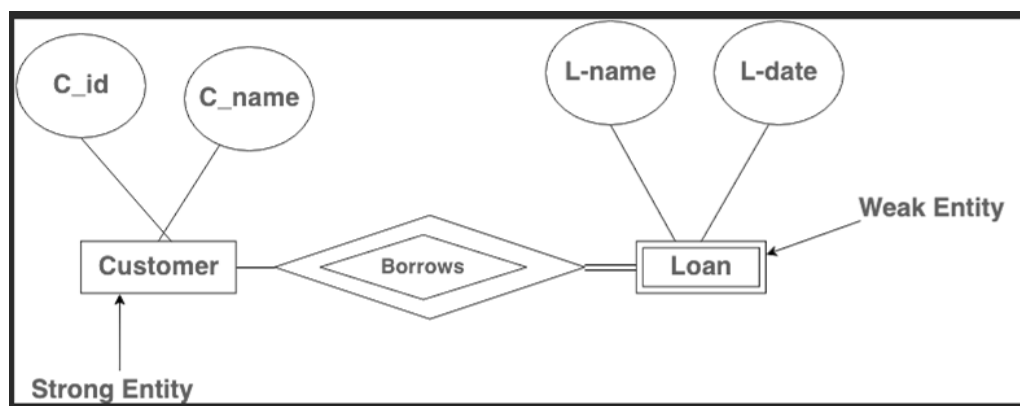
4. Non-Binary Relationship Sets

- **Definition:** Relationships that involve more than two entity sets.
- **Example:** A relationship involving Student, Course, and Instructor where a student enrolls in a course taught by an instructor.
- **Representation:** Shown as a diamond connected to three or more entity rectangles, indicating the participation of all involved entities.

5. Weak Entity Set

- **Definition:** An entity that cannot be uniquely identified by its own attributes alone and relies on a "strong" or "owner" entity.
- **Example:** A Dependent entity may depend on an Employee entity for identification (e.g., DependentName + EmployeeID).
- **Representation:** Weak entities are represented by double rectangles, and their

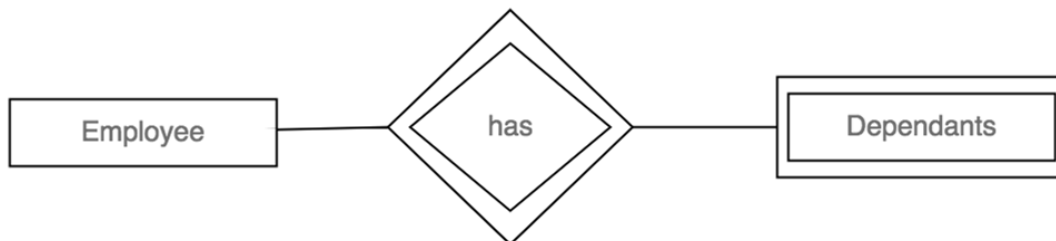
to



relationships strong entities are shown with double diamonds.

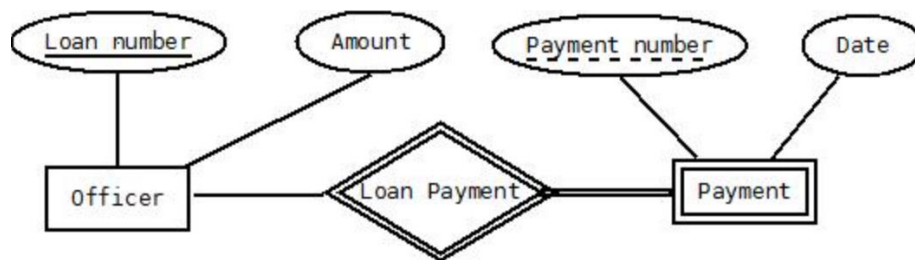
Weak entity is depend on strong entity to ensure the existence of weak entity. Like strong entity, weak entity does not have any primary key, It has partial discriminator key. Weak entity is represented by double rectangle. The relation between one strong and one weak entity is represented by double diamond.

Weak entities are represented with double rectangular box in the ER Diagram and the identifying relationships are represented with double diamond. Partial Key attributes are represented with dotted lines.



Example-1:

In the below ER Diagram, 'Payment' is the weak entity. 'Loan Payment' is the identifying relationship and 'Payment Number' is the partial key. Primary Key of the Loan along with the partial key would be used to identify the records.



Relational Database Design using ER- to Relational Mapping

Relational Database Design using Entity-Relationship (ER) to Relational Mapping is a process that transforms an ER diagram into a relational schema, which can be implemented in a relational database. The process involves converting the different entities, relationships, and attributes represented in an ER diagram into relational tables, keys, and constraints.

Step-by-step approach to converting an ER diagram to a relational schema:

Step 1: Identify Entities and Create Relations

- **Entities** from the ER diagram become **relations (tables)** in the relational model.
- Each **entity** becomes a **table** with **attributes** as columns in that table.
- The **primary key** for each entity is mapped to a **primary key** in the relational model.

Step 2: Handle Multivalued Attributes

- If an entity has **multivalued attributes** (attributes that can have multiple values for a single entity), a **new table** is created to store the multivalued attributes.
- This new table will contain the primary key of the original entity and the multivalued attribute as separate columns. The combination of the primary key of the original entity and the multivalued attribute forms the primary key of the new table.

Step 3: Handle Relationships

- For **one-to-one relationships**:
 - Add the primary key of one of the entities as a **foreign key** in the table of the other entity.
- For **one-to-many relationships**:
 - Add the primary key of the **one side** of the relationship as a **foreign key** in the table of the **many side**.
- For **many-to-many relationships**:
 - Create a new relation (table) to represent the relationship.
 - The table will have **foreign keys** that reference the primary keys of the related entities. The combination of these foreign keys forms a composite primary key for the relationship table.

Step 4: Handle Weak Entities

- **Weak entities** do not have a sufficient unique identifier and depend on a **strong entity** for their identity.
- A new table is created for the weak entity, and the **primary key** of the strong entity is added as a **foreign key** in the weak entity's table. This foreign key, combined with the weak entity's partial key (if any), forms the **primary key** of the weak entity.

Step 5: Define Integrity Constraints

- **Primary Keys**: The primary key of each table is the set of attributes that uniquely identify a record.
- **Foreign Keys**: Foreign keys are added to maintain referential integrity between tables.
- **Check Constraints**: Any additional constraints (like ranges or specific conditions on attributes) should be defined as **check constraints** in the relational schema.

Example

Let's say we have the following ER diagram:

1. **Entities**:
 - **Student**: Attributes: StudentID (Primary Key), Name, Age.
 - **Course**: Attributes: CourseID (Primary Key), CourseName.
2. **Relationships**:
 - **Enrollment** (many-to-many relationship between Student and Course): Attributes: Grade.

Here's how we would map this to relational tables:

1. Student Table:

StudentID (PK)	Name	Age
----------------	------	-----

2. Course Table:

CourseID (PK)	CourseName
---------------	------------

3. Enrollment Table (many-to-many relationship table):

StudentID (FK)	CourseID (FK)	Grade
----------------	---------------	-------

- The **Enrollment Table** has **foreign keys** referencing the primary keys of **Student** and **Course**, and together, these foreign keys form the **composite primary key** for the relationship.

Extended ER Features in RDBMS: Specialization, Generalization, Attribute inheritance, Constraints on generalization, Aggregation

The Extended Entity-Relationship (EER) model introduces additional features to the basic ER model, enabling more complex and nuanced database design. These features are particularly useful when dealing with more intricate real-world scenarios and relationships. Here's a breakdown of these concepts, along with examples:

1. Specialization

Specialization is the process of dividing a higher-level (superclass) entity into lower-level (subclass) entities based on some distinguishing characteristics.

- **Definition:** Specialization is the inverse of generalization. It's the process of creating new subtypes (or subclasses) from an existing entity type, where each subclass contains additional attributes or participates in relationships that are unique to that subclass.
- **Example:**
Consider a **Person** entity with attributes like **PersonID**, **Name**, and **DateOfBirth**. You may want to specialize it into two entities: **Student** and **Employee**, as they have specific attributes and roles that distinguish them.
 - **Person:** PersonID (PK), Name, DateOfBirth
 - **Student** (subclass of Person): StudentID (PK), Degree
 - **Employee** (subclass of Person): EmployeeID (PK), Department

2. Generalization

Generalization is the opposite of specialization. It is the process of combining two or more lower-level (subclass) entities into a higher-level (superclass) entity, grouping common attributes.

- **Definition:** Generalization abstracts similar entities into a more general form, often when they share common characteristics.
- **Example:**
If you have two entities: **Teacher** and **Student**, you might generalize them into a **Person** entity, because they share common attributes such as **PersonID**, **Name**, and **DateOfBirth**.
 - **Teacher:** TeacherID (PK), Subject
 - **Student:** StudentID (PK), Major
 - **Person** (superclass): PersonID (PK), Name, DateOfBirth

Here, **Teacher** and **Student** are specialized types of the **Person** entity.

3. Attribute Inheritance

Attribute inheritance refers to the concept that subclasses inherit attributes from their superclass.

- **Definition:** In an EER model, a subclass automatically inherits all attributes and relationships of its superclass, but it may also have additional specific attributes or relationships.
- **Example:**
In the previous example, if **Person** has an attribute **DateOfBirth**, both **Student** and **Employee** will automatically inherit **DateOfBirth**.
 - **Person:** PersonID (PK), Name, DateOfBirth
 - **Student:** StudentID (PK), Degree (inherits Name, DateOfBirth)
 - **Employee:** EmployeeID (PK), Department (inherits Name, DateOfBirth)

4. Constraints on Generalization and Specialization

Constraints control how generalization and specialization work, ensuring that the model aligns with the real-world conditions.

Types of Constraints:

- **Disjoint vs. Overlapping:**
 - **Disjoint:** An instance of a superclass can belong to **only one subclass**. In other words, an entity cannot belong to multiple subclasses.
 - **Overlapping:** An instance of a superclass can belong to **multiple subclasses**.
- **Total vs. Partial:**
 - **Total:** Every instance of a superclass must be a member of **at least one subclass**.
 - **Partial:** An instance of a superclass may or may not be a member of a subclass.
- **Example of Disjoint and Overlapping:**
 - **Disjoint Example:** A **Person** entity can be either a **Student** or an **Employee**, but not both.
 - **Disjoint Constraint:** A person can be **either** a student **or** an employee, but not both.
 - **Overlapping Example:** A **Person** entity can be both a **Student** and an **Employee** (e.g., a part-time student working as an employee).
 - **Overlapping Constraint:** A person can **be both** a student and an employee.
- **Example of Total vs. Partial:**
 - **Total:** Every **Person** must be either a **Student** or an **Employee** (no person can exist without being one of these).
 - **Partial:** A **Person** may or may not be a **Student** or an **Employee** (a person could be neither).

5. Aggregation

Aggregation is a concept that allows us to treat a relationship set as an entity. This is useful when we need to represent complex relationships where an entity participates in a relationship, which itself has relationships with other entities.

- **Definition:** Aggregation allows a relationship between entities to be abstracted and treated as an entity for the purpose of further relationships.

- **Example:**

Consider a **Project** entity that involves **Employee** and **Department** entities. If an **Employee** works on a **Project**, and each **Project** is associated with a **Department**, we can treat the relationship between **Employee** and **Project** as an entity in itself.

Suppose we have:

- **Employee:** EmployeeID (PK), Name
- **Department:** DepartmentID (PK), DepartmentName
- **Project:** ProjectID (PK), ProjectName
- **WorksOn** (Relationship between **Employee** and **Project**): WorkID (PK), Hours

We can use **Aggregation** to group the relationship between **WorksOn** and **Department**, treating it as a higher-level entity.

- **ProjectAssignment** (aggregated entity): ProjectID (PK), DepartmentID (FK), WorkID (FK)

This allows us to treat the **ProjectAssignment** relationship between **Employee**, **Project**, and **Department** as a separate entity.

